

# LiveKit Open Source vs. LiveKit Cloud for CHAV Telehealth Application

This report outlines the differences between **LiveKit Open Source (Self-Hosted)** and **LiveKit Cloud (SaaS)**, specifically tailored to the feature stack and requirements of the **CHAV Telehealth Application**.

## 1. CHAV Project Feature Stack

Below is the architectural blueprint of the CHAV application:

| CHAV Feature            | Technology               | LiveKit Ecosystem Role                 |
|-------------------------|--------------------------|----------------------------------------|
| **Video Call**          | LiveKit + WebRTC         | Core Media Engine                      |
| **Audio Call**          | LiveKit + WebRTC         | Core Media Engine                      |
| **Waiting Room**        | plugNmeet                | Room Orchestration / Frontend Lobby    |
| **Chat**                | plugNmeet + Socket.IO    | Text/Data Messaging                    |
| **Whiteboard**          | plugNmeet                | Collaborative Canvas                   |
| **File Sharing**        | MinIO + plugNmeet        | Structured Data Transfer               |
| **Live Captions**       | Faster-Whisper           | Local Audio-to-Text Speech Inference   |
| **Sentiment Analysis**  | Hugging Face Models      | Local Text Analysis                    |
| **AI Consultation**     | Pipecat + Ollama + Llama | Local Voice-Agent Orchestration & LLM  |
| **Background Blur**     | MediaPipe                | Client-side Video Processing           |
| **Eye Blink Detection** | OpenCV + MediaPipe       | Client-side Computer Vision            |
| **Noise Suppression**   | RNNoise                  | Client-side Noise Filtering            |
| **Online Status**       | Redis                    | Server-side Presence & Sync            |
| **Notifications**       | Socket.IO                | Real-time Push Messaging               |
| **OTP Verification**    | Node.js + Email OTP      | User Authentication                    |
| **Consent Recording**   | MariaDB                  | Audit Logging & Compliance             |
| **Recording Storage**   | MinIO                    | Self-Hosted S3-Compatible Object Store |
| **Camera Flip**         | Browser API              | Client-side Hardware Control           |
| **Location Sharing**    | Geolocation API          | Client-side Sensors                    |

## 2. Recommendation: LiveKit Open Source (Self-Hosted)

For the CHAV project stack, you **must use LiveKit Open Source (Self-Hosted)**. The key reasons for this requirement are detailed below:

### I. Native Integration with `plugNmeet`

- **The Issue:** Your core client lobby, waiting room, whiteboard, and chat systems are powered by **plugNmeet**.
- **Why it requires Open Source:** plugNmeet is a self-hosted conferencing system designed specifically to deploy and control its own **self-hosted LiveKit Open Source server**. Integrating plugNmeet with LiveKit Cloud requires complex workarounds that defeat the plug-and-play architecture of the plugNmeet system.

## II. Local/Private AI Pipeline

- **The Issue:** You are running **Pipecat + Ollama + Llama** (AI voice consultation) and **Faster-Whisper** (Live Captions).
- **Why it requires Open Source:** Ollama runs LLMs locally on your hardware, and Faster-Whisper performs voice-to-text inference locally. By hosting your own LiveKit Server alongside these services on the same local network or cloud VPC, you keep raw audio/video streams within your private network. This **avoids massive bandwidth egress costs** and **latency spikes** that would occur if you sent streams up to LiveKit Cloud and back to your local AI servers.

## III. Self-Hosted Storage and Database

- **The Issue:** You use **MinIO** for Recording and File Sharing, and **MariaDB** for Consent Recording.
- **Why it requires Open Source:** MinIO is a self-hosted S3-compatible storage system. A self-hosted LiveKit server can record calls and dump them directly into your local MinIO buckets via simple S3 endpoints, without requiring public cloud permissions or exposing storage networks to external SaaS providers.

---

## 3. Service-by-Service Deep Dive: Cloud vs. Self-Hosted

---

Below is a detailed breakdown of how each major component in the LiveKit ecosystem is allocated and operated under both models:

### I. Selective Forwarding Unit (SFU) & Network Routing

- **LiveKit Cloud:**
  - **Architecture:** Runs on a globally distributed **Anycast mesh network**. Users automatically route to the nearest physical edge location.
  - **Cross-Region Sync:** Audio and video streams are synced dynamically across regions via high-speed cloud backbones to reduce latency.
  - **TURN Server:** Automatically managed, globally distributed TURN (Traversal Using Relays around NAT) network ensures firewall traversal.
- **Self-Hosted:**
  - **Architecture:** You deploy the SFU on your own server nodes (e.g., AWS EC2, bare-metal servers).
  - **Scaling & Sync:** Multiple SFU nodes must connect to a shared **Redis** instance to sync room states. Multi-region deployments require you to manage geo-DNS routing and inter-region bandwidth billing.
  - **TURN Server:** You must run and configure a TURN server (typically via `coturn` or LiveKit's built-in TURN option). This requires exposing specific public ports (`3478 UDP/TCP` and `443 TCP` for TLS fallback) to prevent call connection failures for users behind restrictive office or cellular firewalls.

### II. Egress Service (Recording & Streaming)

- **LiveKit Cloud:**
  - **Allocation:** Fully serverless and managed out-of-the-box.
  - **Operation:** Recording a call (Web, Track, or Room Composite) is triggered via simple REST API calls. LiveKit Cloud handles the heavy CPU encoding loads behind the scenes.

- **Storage:** Recordings can be automatically uploaded to any cloud storage bucket (AWS S3, Azure Blob, Google Cloud Storage, or private S3-compatible endpoints like **MinIO**).
- **Self-Hosted:**
- **Allocation:** You must deploy the separate ``livekit-egress`` service container.
- **System Overhead:** Generating a layout recording (Room Composite Egress) spins up a headless **Chromium instance** inside Docker to render the HTML/CSS room layout in real-time.
- **Resource Requirements:** Each simultaneous composite recording job requires **2 to 6 CPUs** and significant memory. Egress workers must be scaled horizontally on dedicated nodes to prevent them from starving the core SFU processes of resources.
- **Docker Security Configuration:** To run Chromium safely, you must pass the ``cap-add=SYS_ADMIN`` flag (to allow Chrome sandboxing) or provide custom Seccomp files, and allocate at least ``shm-size=1g`` to prevent browser crashes.

### III. Ingress Service (Streaming Input via RTMP/WHIP)

- **LiveKit Cloud:**
- **Allocation:** Built-in. You receive an ingress endpoint URL instantly to stream RTMP or WebRTC-based WHIP streams directly into the room.
- **Self-Hosted:**
- **Allocation:** Requires running the separate ``livekit-ingress`` container.
- **Operation:** The ingress container processes inbound RTMP or WHIP feeds, decodes the media, and publishes them as tracks to the LiveKit server.
- **Networking:** Requires exposing specific ports (e.g., ``1935 TCP`` for RTMP and ``7885 UDP`` for WHIP) on your server's public network interface.

### IV. SIP / Telephony Gateway

- **LiveKit Cloud:**
- **Allocation:** Fully managed. You can integrate directly with SIP providers (Twilio, Telnyx) via the LiveKit Cloud dashboard to assign phone numbers to video/audio conference rooms.
- **Self-Hosted:**
- **Allocation:** Requires deploying the ``livekit-sip`` bridge service.
- **Operation:** You must write custom gateway code to translate SIP phone signals to WebRTC tracks, configure outbound trunks, and manage credential handshakes with your chosen telecom provider.

### V. AI Voice Agents (Pipecat / LiveKit Agents)

- **LiveKit Cloud:**
- **Allocation:** Provides managed agent hosting, serverless runtimes, and integrated low-latency AI features (e.g., Whisper-level cloud noise suppression).
- **Self-Hosted:**
- **Allocation:** You write agent workers using the LiveKit Python/Node.js SDKs (configured with **Pipecat** and local engines like **Ollama**).
- **Operation:** Your agent workers run on your own GPU/CPU servers. They register with the shared Redis

instance, wait for room events (such as a participant joining), and dynamically connect to the local SFU.

- **Benefit:** Zero data transmission cost since the AI agents, the LLM engine (Ollama), and the LiveKit media streams run inside the same network perimeter.

## VI. Observability & Analytics

- **LiveKit Cloud:**
  - **Allocation:** Native web dashboard. Visualizes real-time metrics including jitter, packet loss, round-trip time, bitrate, and connection types for every participant.
- **Self-Hosted:**
  - **Allocation:** The LiveKit server exports standard **Prometheus metrics** at a `/metrics`` endpoint.
  - **Operation:** You must build your own monitoring stack: deploy Prometheus to scrape the metrics, store them in a database, and configure Grafana dashboards to visualize the health of your SFU clusters.

## 5. Technology Comparison Matrix

| Category                | LiveKit Cloud               | Self-Hosted (Open Source)                    | CHAV Stack Fit                                               |
|-------------------------|-----------------------------|----------------------------------------------|--------------------------------------------------------------|
| **Deployment & Ops**    | Managed SaaS                | Self-hosted (Docker, Kubernetes)             | **Self-Hosted** fits your local MinIO/MariaDB stack          |
| **Network Egress Cost** | Charged per-minute/gigabyte | Pay only for raw VM bandwidth                | **Self-Hosted** prevents high costs from local AI processing |
| **TURN/STUN Setup**     | Automated (Global Edge)     | Manual configuration (e.g. `coturn`)         | **Self-Hosted** requires setting up a TURN server            |
| **Egress (Recordings)** | Managed Serverless          | Requires separate `livekit-egress` container | **Self-Hosted** matches your private **MinIO** storage       |
| **Inference/AI**        | Cloud LLM/TTS/STT           | Custom GPU/CPU Workers                       | **Self-Hosted** is native to your **Ollama/Pipecat** setup   |
| **Observability**       | Cloud Dashboard             | Prometheus / Grafana exporter                | **Self-Hosted** requires manual monitoring config            |

## 6. Setup Instructions

### I. Self-Hosted Setup (Recommended for CHAV)

To self-host, you deploy the LiveKit server components (SFU) onto your own local machine or private cloud.

#### Local Development Mode

#### 1. Download the Binary:

Download the appropriate executable from the [LiveKit Github Releases](#).

#### 2. Start in Development Mode:

Execute the following command (similar to your current terminal command):

```
./livekit-server --dev --bind 0.0.0.0 --node-ip <YOUR_LOCAL_IP>
```

- `bind 0.0.0.0` allows the server to listen on all interfaces.
- `node-ip` specifies the local IP address (e.g., `10.21.95.252`) that client applications will use to connect.

### 3. Lobby Credentials:

By default, `dev` mode generates a placeholder key and secret:

- **API Key:** `devkey`
- **API Secret:** `secret`

#### Production Mode (Docker Deployment)

#### 1. Prepare the `livekit.yaml` config:

```
port: 7880
rtc:
  port_range_start: 50000
  port_range_end: 60000
  use_external_ip: true
redis:
  address: redis:6379
keys:
  devkey: secret # Match your plugNmeet keys
```

#### 2. Deploy using Docker Compose:

```
version: '3.9'
services:
  livekit:
    image: livekit/livekit-server:latest
    command: --config /etc/livekit.yaml
    network_mode: "host"
    volumes:
      - ./livekit.yaml:/etc/livekit.yaml
    restart: always
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

### 3. Firewall Configuration:

Open the following ports on your host/firewall:

- `7880 TCP` (HTTP/WS API)
- `443 TCP` (HTTPS/WSS if terminated at host)
- `50000-60000 UDP` (WebRTC media streams)
- `3478 UDP` (TURN server)

## II. LiveKit Cloud Setup

If you ever choose to migrate to LiveKit Cloud:

1. **Sign up** for an account on [livekit.io](https://livekit.io).
2. **Create a Project** in the Cloud dashboard.
3. **Retrieve API Keys** (API URL, API Key, and API Secret).
4. **Connect:** Update the LiveKit connection URL in your client configuration to ``wss://your-project-id.livekit.cloud``.